

ezr, week 0: the port, warm-up

the story

All of 101.py, verbatim, with glossary notes folded in (click a to open one) and this week's exercises at the bottom.

Nothing clever yet: one small Python shell — settings from a help string, tests run from the command line, seeds reset before every run. Every later week's port grows inside this file.

python docstrings (**doc**), SSOT (single source of truth), mechanism-policy I

A string as the first statement of a file (or def) is stored, not executed: `__doc__`. 101.py's docstring is its usage message (`-h` just prints it) AND its settings table — `settings(__doc__)` regex-scrapes the defaults out of the help text. One string: help, defaults, documentation. That is SSOT and mechanism-policy in thirteen lines of Python.

Say each fact once; derive everything else. E.g. define the options ONCE, in a help string; parse settings out of that string; then code and documentation can never drift apart:

Other SSOTs here: the README schedule (all dates), row 1 of a csv (all column roles). SSOT is mechanism-policy's best friend: the single source is the policy. Separate the *what* (policy: small, declarative, easy to change) from the *how* (mechanism: code that obeys any policy). Then one mechanism serves a thousand policies. Everywhere in this course:

policy (a little data)	mechanism (code)
<code>__doc__</code> options text in 101.py row 1 of a csv (Lbs-, Acc+...)	the regex that parses it the csv reader, Cols, heaven, disty
keys of the eg demo table the settings table	the <code>go()</code> dispatcher every function reading it

Change the policy line, never the mechanism: a new dataset is a new header row, not new code.

```
"""
101.py: just enough to reset settings from the command line
(c) 2026, Tim Menzies <timm@ieee.org>, MIT license

USAGE: python3 101.py [--key=val ...] [test ...]

OPTIONS: (defaults below are parsed into `the`):
--file   data file = $MOOT/optimize/misc/auto93.csv
--seed   random seed      = 1
--round  decimals shown   = 3
-h       print this help
"""

import os, re, random, sys; sys.dont_write_bytecode = True
from math import floor
from types import SimpleNamespace as o
```

python environ I

`os.environ` is a dict of the shell's variables. Lets one default live outside the code, per machine:
Set `MOOT=/somewhere` in your shell and `101.py` finds your data; set nothing and a sane default fires.

```
MOOT = (os.environ.get("MOOT") or  
        os.path.expanduser("~/gits/moot"))
```

python f-strings I

`f"..."` runs the `{...}` parts as code; after a `:` comes a format spec, which may itself be `{computed}`:
That second line is why `--round=4` changes every number 101.py prints: one policy value, one printing mechanism.

```
def say(x):  
    if isinstance(x, float):  
        return (f"{x:.0f}" if x==floor(x) else f"{x:.{the.round}f}")  
    if isinstance(x, dict): return "{ " + " ".join(  
        f":{k} {say(v)}" for k,v in x.items())if str(k)[0]!="_"+"}"  
    return str(x)
```

python slices I

`x[lo:hi]` is items `lo` to `hi-1`; blanks mean "from the start" or "to the end"; negatives count from the end:

```
x = [a, b, c, d, e]
      0 1 2 3 4      <- index
     -5 -4 -3 -2 -1  <- negative index
```

```
x[:2] = [a, b]      x[2:] = [c, d, e]
x[-2:] = [d, e]     x[1:3] = [b, c]
```

`x[:]` = a copy of `x`

In 101.py: `s[:1]`=="-" (first char), `s[1:]` (the rest), `a[2:]` (strip a leading --).

```
def thing(s):
    if (s[1:] if s[:1]=="-" else s).isdigit(): return int(s)
    try: return float(s)
    except ValueError: return s=="True" or (s!="False" and s)
```

regx (regular expressions) I

Little languages for matching text. Just enough for 101.py and the Lua at its side
(Lua patterns use % where Python uses \, and - where Python uses *?):

means	python	lua
word char (letter/digit)	\w	%w
whitespace / non-space	\s \S	%s %S
letter / lowercase	[a-zA-Z]	%a %l
any chars, lazy	.*?	.-
start / end of string	^ \$	^ \$
one char from a set	[^=\n]	[+~]
capture a group	()	()
all matches	re.findall	s:gmatch
find / replace	re.search, re.sub	s:find, s:gsub
a literal .	\.	%.

The two worked examples, one per language — 101.py scraping --key ... =
default pairs from its docstring, ezr picking a column's kind off its first letter:

```
return (name:find"~%l" and Sym or Num)(name,at) -- ezr.lua
```

```
def settings(doc):  
    pat = r"--(\w+)\s+[^=\n]*=\s*(\S+)"  
    return o(**{k: thing(v) for k,v in re.findall(pat, doc)})  
  
def test_config(): print(say(vars(the)))
```

python argv, RNG (random number generator), TDD (test-driven development) I

sys.argv is the command line, split on spaces: argv[0] the script name, the rest yours. 101.py walks it twice — first pass updates settings from --key=val, second runs any named tests:

Note that last line; see RNG, next.

Computers do not roll dice. An RNG is a deterministic formula that *looks* random; from the same seed, the same stream, forever. This course uses the Park-Miller minimal standard (one multiply, one modulo):

```
Seed = (16807 * Seed) % 2147483647
```

The point of need: RESET the seed before every run. Then every experiment replays exactly — same seed, same “random” numbers, same result — on your machine, your grader’s, and in Lua or Python alike (ports are graded by diff-ing the two streams). 101.py does this before every test:

Forget the reset and your “bug” changes every run. Reset, and science becomes repeatable.

Red, green, refactor: write a failing test (red), write just enough code to pass (green), then clean up with the tests as a safety net (refactor). This code’s dialect: every demo in the eg files reseeds, prints, then asserts — no crash means pass — and the harness never dies mid-suite. run traps a failing demo and prints its stack dump, so --all can count failures and keep going:

```
function run(funs,w,    ok,msg)
  srand(the.seed)
  if funs[w] then
    ok, msg = xpcall(funs[w], debug.traceback)
    if not ok then print(msg) end
    return ok end end
```

The eg table is the whole test framework: demos are just entries, so adding a test is one assignment, no registration ceremony:

```
eg = {}          -- the demo table
```

```
eg["--col"] = function(    n)  -- green: watch, then lock in
  n = adds{1,2,3,4,5}
  print(show{mu=n:mid(), sd=n:div()})
  assert(n:mid() == 3) end
```

```
eg["--ent"] = function(    s)
  s = adds({"a","a","b"}, Sym())
  assert(abs(s:div() - 0.918) < 0.01) end
```

```
eg["--broke"] = function()  -- red: prints a stack dump;
  assert(2 + 2 == 5) end    -- --all counts it. moves on
```

```
def main(funs):
    if "-h" in sys.argv: return print(__doc__)
    for a in sys.argv[1:]:
        if a[:2]=="--" and "=" in a:
            k,v = a[2:].split("=",1)
            if k in vars(the): setattr(the, k, thing(v))
    for a in sys.argv[1:]:
        if (n := "test_"+a) in funs:
            random.seed(the.seed); funs[n]()

the = settings(__doc__)

if __name__=="__main__": main(globals())
```


exercises I

Exercises, week 0

1. Run `python3 101.py config`, then again with `--round=1 --seed=42 config`. Explain every difference.
2. Add `test_rand`: print ten random numbers. Run it twice with the same seed, then twice with different seeds. What is guaranteed, and by which line of `main`?
3. Port `rand.lua` (Park-Miller) to Python. Same seed, SAME 20 numbers as the Lua, or the port is wrong: `diff <(lua rand.lua) <(python3 rand.py)` prints nothing.
4. Add one new option to the docstring only (say `--bins cuts = 5`). Prove the `.bins` now exists without touching any code. Which principle is that?
5. Break thing: what does it return for `"-3"`, `"3.1.4"`, `"None"`? Which answers surprise you?